

Calculus for Machine Learning

Derivatives, Gradients & the Chain Rule — Explained, With Examples and ML Applications

Part of the 3-Month Machine Learning Mentorship Program

Training a machine learning model is, underneath everything else, an optimization problem: find the parameters that make a loss function as small as possible. Calculus is the toolkit for doing exactly that. This guide walks through the three pieces that matter most — derivatives, gradients, and the chain rule — with worked examples and a direct line drawn to how each one shows up in real ML systems.

1. Why Calculus Matters for ML

Every model you train — from a simple linear regression to a large neural network — learns by adjusting its parameters to reduce a loss function. The question calculus answers is simple: *given the current parameters, which direction should each one move to reduce the loss, and by how much?*

- **Derivatives** tell you the slope of the loss at your current parameters — which way is “downhill.”
- **Gradients** generalize derivatives to many parameters at once — a whole direction to move in.
- **The chain rule** lets you compute gradients through deeply layered functions — this literally is backpropagation.

The core idea: Nothing in this guide is calculus for its own sake. Every concept exists to answer one question: how do we nudge a model's parameters to make it a little more correct?

2. Derivatives

2.1 What Is a Derivative?

A derivative is the **instantaneous rate of change** of a function — how fast its output changes as you nudge the input, measured at a single exact point. Geometrically, it's the slope of the line tangent to the

function's curve at that point.

It's written $f'(x)$ or dy/dx , and both mean the same thing: “the derivative of the function.” A steep curve has a derivative with a large absolute value; a flat curve (like at the bottom of a bowl shape) has a derivative near zero — which is exactly how we recognize a minimum or maximum.

2.2 The Rules of Differentiation

A handful of rules cover most of what you'll need:

Rule	Formula	Example
Power rule	$d/dx[x^n] = n \cdot x^{n-1}$	$d/dx[x^3] = 3x^2$
Constant multiple	$d/dx[c \cdot f(x)] = c \cdot f'(x)$	$d/dx[5x^2] = 10x$
Sum rule	$d/dx[f(x)+g(x)] = f'(x)+g'(x)$	$d/dx[x^2+3x] = 2x+3$
Constant	$d/dx[c] = 0$	$d/dx[7] = 0$

2.3 Worked Example: Derivative of a Loss Function

Suppose a tiny model predicts $y_hat = w \cdot x$ for a single training example with $x=2$ and a true value $y=10$. The squared-error loss is $L(w) = (y - w \cdot x)^2$. Differentiating with respect to w tells us exactly how to adjust it:

```
import sympy as sp
w = sp.symbols('w')
x_val, y_val = 2, 10
L = (y_val - w * x_val) ** 2

dL_dw = sp.diff(L, w)
print(dL_dw)           # -4*(-2*w + 10)
print(dL_dw.subs(w, 3)) # -16 -- the slope at w=3
```

At $w=3$, the slope is -16 — negative, meaning *increasing* w will *decrease* the loss. That single number is the entire signal gradient descent needs to know which way to move. Solving $dL/dw = 0$ gives $w=5$, the exact minimum.

3. Gradients

3.1 Partial Derivatives

Real models have many parameters, not just one. A **partial derivative** measures the rate of change with respect to just one variable, holding all the others fixed — written $\partial f / \partial x$. For $f(x, y) = x^2 + y^3$, the

partial with respect to x treats y as a constant: $\partial f / \partial x = 2x$. The partial with respect to y treats x as a constant: $\partial f / \partial y = 3y^2$.

3.2 The Gradient Vector

$$\nabla f = [\partial f / \partial x_1, \partial f / \partial x_2, \dots, \partial f / \partial x_n]$$

- The gradient collects every partial derivative into a single vector.
- It points in the direction of steepest **increase** of the function.
- Gradient descent moves the **opposite** way — subtracting the gradient — to decrease loss as fast as possible.
- Every parameter in a model gets its own entry in this vector, updated simultaneously each training step.

3.3 Gradient Descent

Gradient descent is the algorithm that actually uses the gradient: start with a random guess for the parameters, compute the gradient of the loss, take a small step in the opposite direction, and repeat. Picture the loss as a bowl-shaped surface — gradient descent walks downhill, taking smaller steps automatically as the surface flattens out near the bottom.

```
def f(x):  
    return (x - 3) ** 2      # minimum is at x = 3  
  
def f_prime(x):  
    return 2 * (x - 3)  
  
x = 0.0  
learning_rate = 0.1  
for step in range(20):  
    grad = f_prime(x)  
    x = x - learning_rate * grad    # move OPPOSITE the gradient  
  
print(x)    # approaches 3.0
```

On the learning rate: The learning rate scales how big each step is. Too small and training crawls; too large and parameters can overshoot the minimum and diverge instead of converging.

4. The Chain Rule

4.1 The Formula and the Intuition

$$d/dx[f(g(x))] = f'(g(x)) \cdot g'(x)$$

Whenever one function is nested inside another — a **composite function** — the chain rule differentiates the whole thing by multiplying the derivatives of each piece together. This shows up constantly: almost every interesting function in ML is a composition of simpler ones layered together.

4.2 Worked Example

Take $f(x) = (3x + 1)^2$. This is really two nested functions:

- inner: $g(x) = 3x + 1$
- outer: $f(u) = u^2$

By the chain rule: $f'(x) = 2 \cdot (3x+1) \cdot 3 = 6 \cdot (3x+1)$.

```
import sympy as sp
x = sp.symbols('x')
f = (3*x + 1)**2
print(sp.diff(f, x))    # 6*(3*x + 1) -- matches the hand calculation
```

4.3 The Chain Rule Is Backpropagation

A neural network is a deep composite function: $\text{input} \rightarrow \text{layer 1} \rightarrow \text{layer 2} \rightarrow \dots \rightarrow \text{loss}$.

Backpropagation is nothing more than the chain rule, applied layer by layer, walking backward from the loss to every weight in the network:

```
# conceptually, for Loss = f3(f2(f1(x))):
d_loss_d_w1 = d_loss_d_f3 * d_f3_d_f2 * d_f2_d_f1 * d_f1_d_w1
#           (this whole chain of multiplications IS backpropagation)
```

Every weight update in every layer of every neural network you'll ever train is computed this way — a chain of local derivatives, multiplied together, exactly as shown above.

5. Real-World ML Applications

Training Neural Networks

Every weight update, in every layer, is computed via gradients and the chain rule (backpropagation). This is the mechanism, not an approximation of it.

Logistic Regression

Finding the best-fit parameters means minimizing a loss function with gradient-based optimization — the same core loop as the gradient descent example above.

Learning Rate Tuning

The learning rate scales how big each gradient-descent step is. Tuning it well is often the single highest-leverage hyperparameter decision in training a model.

Modern Optimizers (Adam, RMSprop, etc.)

These build on plain gradient descent with smarter, adaptive step sizes per parameter — but gradients, computed via the chain rule, remain the foundation underneath all of them.

6. Summary & Cheat Sheet

Concept	Meaning	Formula
Derivative	Instantaneous rate of change / slope of the tangent line	$f'(x)$ or dy/dx
Partial derivative	Rate of change w.r.t. one variable, others held fixed	$\partial f / \partial x$
Gradient	Vector of all partial derivatives; points toward steepest increase	$\nabla f = [\partial f / \partial x, \partial f / \partial y, \dots]$
Gradient descent	Repeatedly step opposite the gradient to minimize a function	$x = x - \alpha \cdot \nabla f(x)$
Chain rule	Differentiates nested (composite) functions	$d/dx[f(g(x))] = f'(g(x)) \cdot g'(x)$
Backpropagation	The chain rule, applied layer by layer through a network	(chained local derivatives)

The one-sentence version: A derivative is a slope, a gradient is a vector of slopes, gradient descent uses that vector to walk downhill, and the chain rule is what makes that possible through as many layers as you want to stack.

7. Further Resources

- **3Blue1Brown** — “**Essence of Calculus**” (YouTube, free): the best visual intuition for everything in this guide.
- **Companion slide deck:** `Calculus_for_ML_DeepDive.pptx` — the same material, presented visually with diagrams.
- **Companion code:** 01–06 in the accompanying code folder — every example above, runnable and extendable.

Derivatives, gradients, and the chain rule aren't abstract math for their own sake — they're the literal mechanism by which every model you train learns.